- Library Automation System (QR-Based)
  - 🛠️ Tech Stack
  - 💻 Prerequisites & Setup Requirements
  - 🚀 1. Backend Server & Admin Panel Setup
    - Step-by-Step Installation:
  - 📱 2. Student Android App Setup (Using Expo & Android Studio)
    - Connecting the App to the Backend
    - Running with Expo Go (Easiest Method for Quick Testing)
  - 🏗️ 3. Importing and Building in Android Studio (Native APK)
    - Step 3.1: Generate the Native Android Folder
    - Step 3.2: Opening in Android Studio
    - Step 3.3: Gradle Sync & Building
  - 📂 Project Structure Guide
  - 🧠 Features Walkthrough

Library Automation System (QR-Based)

A comprehensive, hybrid library management solution comprising a **Node.js Web-based Admin Portal** and a **React Native (Expo) Android App** for students. It facilitates instant book retrieval, QR-based checkouts, and seamless inventory tracking.

🛠️ Tech Stack

- **Backend:** Node.js, Express.js, SQLite3, Socket.IO (for real-time updates)
- **Admin Panel:** HTML5, Vanilla JavaScript, CSS3 (Glassmorphism design)
- **Student App:** React Native, Expo, React Native Camera Network, Async Storage

---

💻 Prerequisites & Setup Requirements

Before you begin, ensure you have the following installed on your machine:

1. **Node.js** (v18.0 or newer)
2. **Git**

3. **Android Studio** (Required for compiling and running the native Android app)
4. A physical Android device or an Android Emulator set up via Android Studio.

🚀 1. Backend Server & Admin Panel Setup

The core backend system acts as both the database host and the server for the admin web panel.

Step-by-Step Installation:

1. **Clone the repository** (if you haven't already) or open the project folder in your terminal.
2. In the root directory (where `server.js` is located), open your terminal.
3. **Install Dependencies:** Run the following command to download all required modules:

   ```
   npm install
   ```

4. **Start the Server:**

   ```
   node server.js
   ```

   (Wait until the terminal outputs: `Server running on http://localhost:3000`)

5. **Access the Admin Panel:** Open any web browser and go to: `http://localhost:3000`
   - Default Login credentials:
     - **Email:** `admin@library.local`
     - **Password:** `admin123`

📱 2. Student Android App Setup (Using Expo & Android Studio)

The student application is built using React Native via the Expo framework.

Connecting the App to the Backend

Before building the app, make sure it points to your computer's IP address (if testing locally) or your deployed URL (e.g., Render/Heroku).

1. Navigate to `/app/src/api.js`.
2. Update the `API_BASE` variable to match your network or hosting URL.

Running with Expo Go (Easiest Method for Quick Testing)

If you just want to test the app without compiling native Android code:

1. Open a new terminal and navigate to the `app` folder:

   ```
   cd app
   ```

2. Install dependencies:

   ```
   npm install
   ```

3. Start the Expo development server:

   ```
   npx expo start --clear
   ```

4. Install the **Expo Go** app on your physical Android phone from the Google Play Store, and scan the QR code that appears in your terminal.

🏗️ 3. Importing and Building in Android Studio (Native APK)

If you want to compile the raw Java/Kotlin code, build a standalone `.apk`, or run the app on an Android Studio Emulator, follow this procedure carefully:

Step 3.1: Generate the Native Android Folder

React Native (Expo) requires you to generate native files before opening them in Android Studio.

1. Open terminal in the `app` directory.
2. Run the `prebuild` command:

   ```
   npx expo prebuild --platform android
   ```

   This command creates a new folder named `/android` inside your `app` directory.

Step 3.2: Opening in Android Studio

1. Open **Android Studio**.
2. Click on **"Open"** or **"Open an existing project"**.
3. Navigate to your project directory. Expand the `app` folder and specifically select the newly created **`android`** folder (`Library Automation System QR/app/android`).
4. Click **OK**.

Step 3.3: Gradle Sync & Building

1. Once opened, Android Studio will automatically begin **"Gradle Sync"** (downloading Android SDKs, libraries, and Java dependencies). Wait for the loading bar at the bottom right to finish.
2. Ensure you have a valid SDK installed. (If Android Studio prompts you to update Gradle or install missing SDKs, accept them).

3. **To Run on an Emulator:**
    - Click the **Device Manager** icon.
    - Create or select a Virtual Device (e.g., Pixel 6 API 33).
    - Click the green **Play (Run)** button at the top toolbar.
4. **To generate an APK file:**
    - In Android Studio, go to the top menu bar.
    - Click **Build** > **Build Bundle(s) / APK(s)** > **Build APK(s)**.
    - Wait for the build process to finish. Once done, a popup will appear at the bottom right saying "Build APK successfully". Click on **"locate"** to find your output `app-debug.apk` file.

📂 Project Structure Guide

- `/server.js` - Main backend logic, Express routes, and Socket.IO configuration.
- `/library.db` - Automatically generated SQLite database file.
- `/public/` - Contains all HTML, CSS, and JS files for the Web Admin Portal.
- `/app/` - The React Native student application workspace.
    - `/app/App.js` - Main entry point and UI screens for the student app.
    - `/app/src/api.js` - Contains fetch calls communicating with the backend.
    - `/app/android/` - (Generated) Native Android source code for Android Studio.

🧠 Features Walkthrough

1. **Intelligent Booting:** Server creates an empty database if it doesn't exist upon start.
2. **Instant QR Scanning:** Students can scan the back of a physical book; a real-time request is pushed to the Admin portal immediately.
3. **Admin Dashboard Analytics:** Total issued books, total active requests, and student history tracking.
4. **Offline Batch Rendering:** The Admin can generate PDFs of QR codes to be printed and pasted on books physically.

Developed for efficient academic and institutional library management.